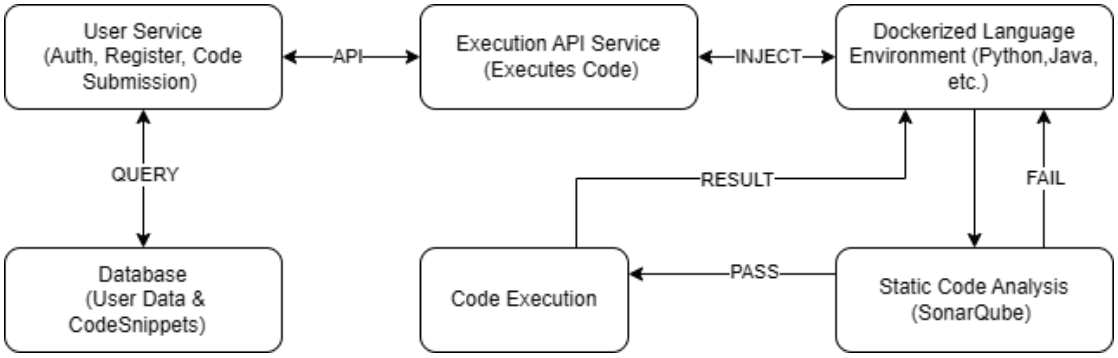
Project: Online Code Execution Platform

System Design:

The system is designed as a microservices architecture to handle user registration, authentication, code submission, execution, and sharing. The architecture includes two main Java-based microservices, which interact with Docker containers for language-specific code execution and static code analysis.

System Architecture:



Components:

1. **User Service Microservice:**
   - Manages user registration, login (using JWT), and code submission.
   - Stores user data and code snippets in a MySQL database.
   - Handles the email verification process to ensure the authenticity of user registration (future).
   - forwards the request to Execution API Service with code and language
2. **Execution API Service:**
   - Injects the request to the Dockerized environment for execution.
   - Capture's the execution log and send back in response
   - This service is stateless, horizontally scalable and placed behind a load balancer to manage high concurrency.
3. **Dockerized Language Environments:**
   - Each language (Python, Java, etc.) runs in a dedicated Docker container.
   - SonarQube runs inside each container to perform static code analysis before executing the code to ensure quality and security.

Database Design:

The system uses MySQL for managing relational data with the following tables:

Users Table: Stores information about users including authentication details.

Field	Type	Description
user_id	**INT**	**Primary Key, Auto Increment**
username	**VARCHAR(255)**	**Unique Username**
password_hash	**VARCHAR(255)**	**Secure password hash**
email	**VARCHAR(255)**	**User's email address**
is_enabled	**BOOLEAN**	enabled **after email is verified**

CodeSnippets Table: Stores code snippets submitted by users. Includes a shareable flag to control which code can be shared**.**

Field	Type	Description
code_id	INT	Primary Key, Auto Increment
user_id	INT	Foreign Key to Users table
language	VARCHAR(50)	Programming language (Python, Java, etc.)
code	TEXT	Actual code snippet
result	TEXT	Execution result or error
timestamp	DATETIME	Submission timestamp
shareable	BOOLEAN	Flag to mark if code can be shared

EmailVerification Table: Stores email verification data for user registration. (future)

Field	Type	Description
verification_id	INT	Primary Key, Auto Increment
user_id	INT	Foreign Key to Users table
verification_token	VARCHAR(255)	Token sent via email for verification
verified_at	DATETIME	Timestamp when email was verified

Key APIs:

1. **Endpoint: /auth/register**
   - Method: POST
   - Description: Registers a new user and sends an email verification link.
   - Request: JSON object with username, email, and password.
   - Response: Success or error message. If successful, an email verification link is sent.
2. **Endpoint: /auth/login**
   - Method: POST
   - Description: Authenticates a user using username and password then returns JWT Token.
   - Request: JSON object with username and password.
   - Response: JSON object containing JWT token for authenticated requests.
3. **Endpoint: /auth/verify-email (future)**
   - Method: GET
   - Description: Verifies the user's email using the verification token.
   - Request: URL parameter with token.
   - Response: JSON object confirming email verification.
4. **Endpoint: /execute/code**
   - Method: POST
   - Description: Submits code for execution.
   - Request: JSON object with language, code, and JWT token for authentication.
   - Response: JSON object with execution results or error message.

5. **Endpoint: /code/{code_id}/share**
    - Method: POST
    - Description: Marks a code snippet as shareable.
    - Request: URL parameter with token.
    - Response: Success message indicating the code is now shareable.
6. **Endpoint: /code/{code_id}/unshare**
    - Method: POST
    - Description: Marks a code snippet as not shareable.
    - Request: URL parameter with token.
    - Response: Success message indicating the code is no longer shareable.
7. **Endpoint: /code/**
    - Method: GET
    - Description: Retrieves a list of code_id along the timestamp that the user has submitted .
    - Request: with JWT token.
    - Response: JSON array of object, including code_id, language, timestamp.
8. **Endpoint: /code/{code_id}**
    - Method: GET
    - Description: Retrieves a specific code snippet by code_id if shareable flag is enabled of its submitted by the user.
    - Response: JSON object with code snippet details (language, code, result, timestamp).

Overall Approach:

The system leverages microservices to ensure scalability, security, and maintainability. Key decisions and technologies are outlined below:

- **Backend Framework**: Java (Spring Boot) for building RESTful microservices, which makes the system easy to maintain, scale, and integrate with Docker and JWT-based authentication.
- **JWT for Authentication**: We use JWT tokens for stateless authentication, enabling secure user login without storing sensitive session data on the server. Each request requires a valid JWT token to authenticate the user.
- **Email Verification**: Users must verify their email addresses during registration. An email verification link with a unique token is sent to the user. Only after successful verification can the user access the platform (**future**) .
- **Database Design**: We use MySQL for relational data storage. The shareable flag in the CodeSnippets table allows users to control which of their code snippets can be shared with others.
- **Dockerized Execution Environments**: Each programming language runs in a dedicated Docker container to isolate code execution environments. SonarQube runs inside these containers to perform static code analysis before execution, ensuring code quality and security.
- **Horizontal Scaling**: The Execution API Service is designed to be stateless, enabling it to scale horizontally. This ensures the system can handle high traffic volumes during peak times using load balancing.
- **Security and Rate Limiting**: The system implements rate limiting and JWT-based security to protect against abuse and unauthorized access, ensuring that the platform is secure and responsive.

Conclusion:

This Online Code Execution Platform is designed for flexibility, scalability, and security, enabling users to execute and share code in a safe and efficient manner. By leveraging microservices, Docker, and JWT-based authentication, the system is capable of handling high concurrency while ensuring a smooth user experience.

The modular architecture allows for future enhancements, such as the integration of additional features, languages, or advanced security measures, ensuring the platform can grow as user demand increases.